PTD-P

Danyang Zhuo
Duke University

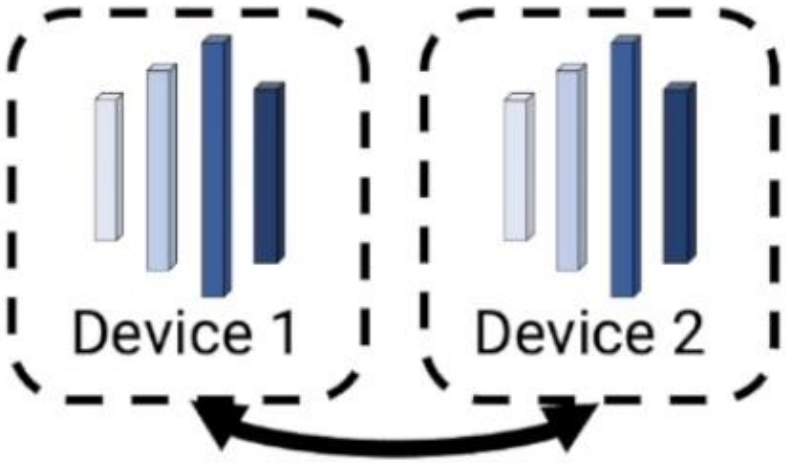
Some materials Jared Casper’s slides

Programming Assignment #2

- <0.2x speedup: grade subject to your implementation
- 0.2 - 0.27x speedup: 60 / 100 points
- 0.27 - 0.32 x speedup: 80 / 100 points
- 0.32x speedup and above: 100 / 100 points

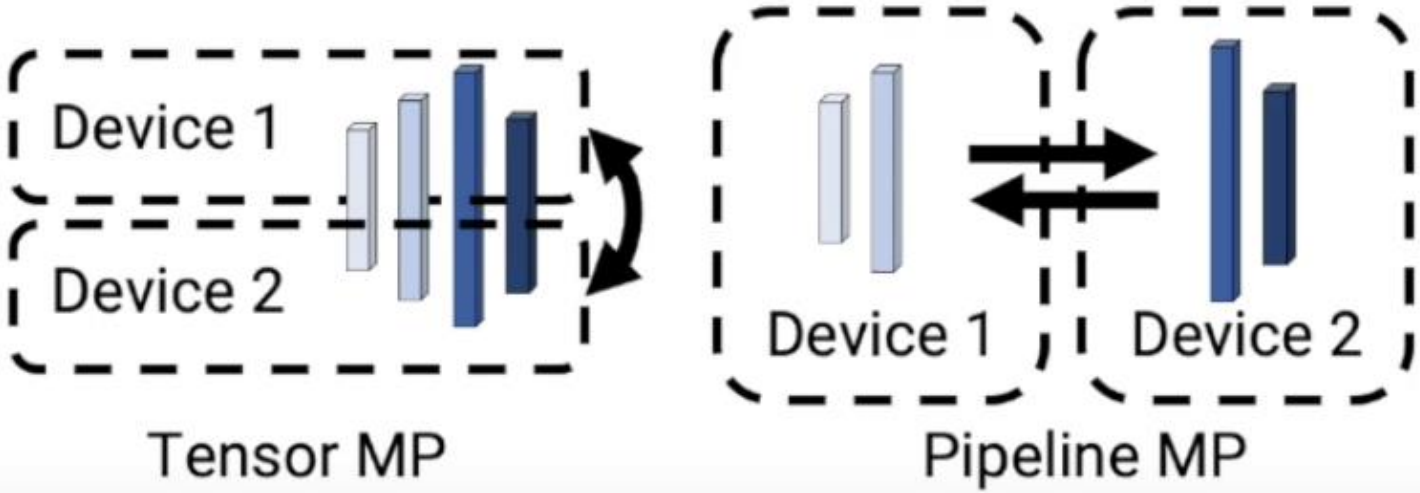
PARALLEL TRAINING

Data Parallelism (DP)



n copies of model parameters

Model Parallelism (MP)

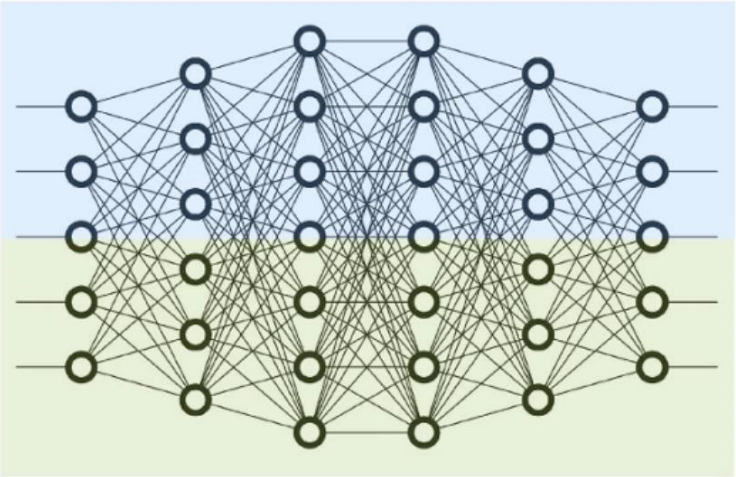


Single copy of model parameters

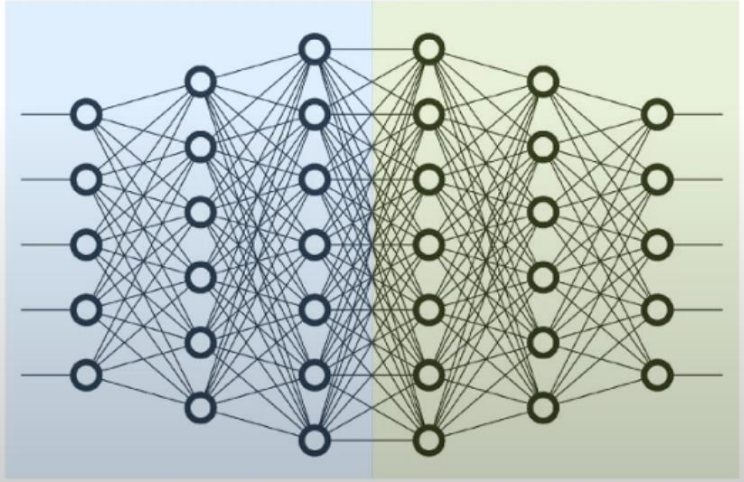
MODEL PARALLELISM

Pipeline (Inter-Layer) Parallelism

- Tensor (Intra-Layer) Parallelism
  - Split individual layers across multiple devices
  - Both devices compute difference parts of Layer 0,1,2,3,4,5



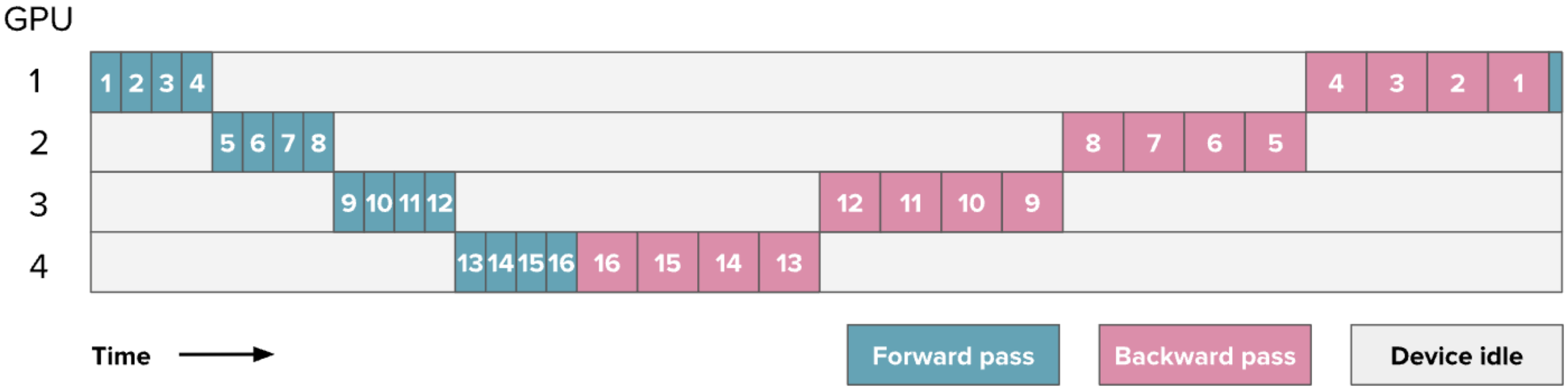
- Pipeline (Inter-Layer) Parallelism
  - Split sets of layers across multiple devices
  - Layer 0,1,2 and layer 3,4,5 are on difference devices



High-level picture

- How should parallelism techniques be combined to maximize the training throughput of large models given a batch size while retaining strict optimizer semantics?

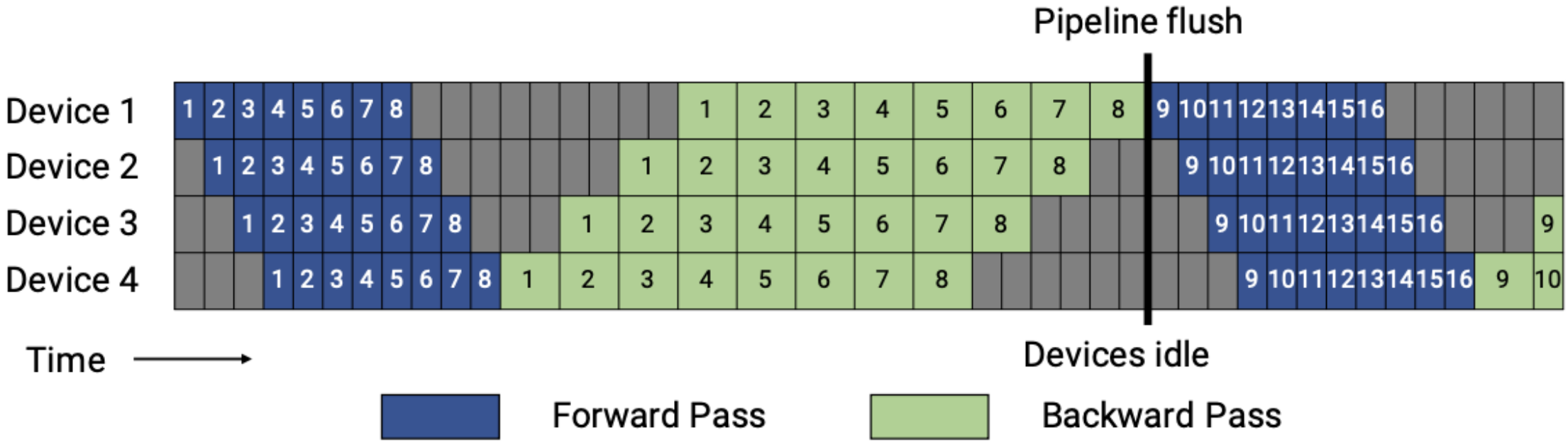
- Combination of pipeline parallelism across multi-GPU servers, tensor parallelism within a multi-GPU server, and data parallelism, to practically train models with a trillion parameters with graceful scaling in an optimized cluster environment with high-bandwidth links between GPUs on the same server and across servers.



An example of Pipeline parallelism for a model with 16 layers distributed across 4 GPUs. The numbers correspond to the layer IDs.

from “The Ultra-Scale Playbook: Training LLMs on GPU Clusters”

GPipe



This is called all-forward, all-backward (afab)

Question: Why not keep scaling up #micro-batches per batch?

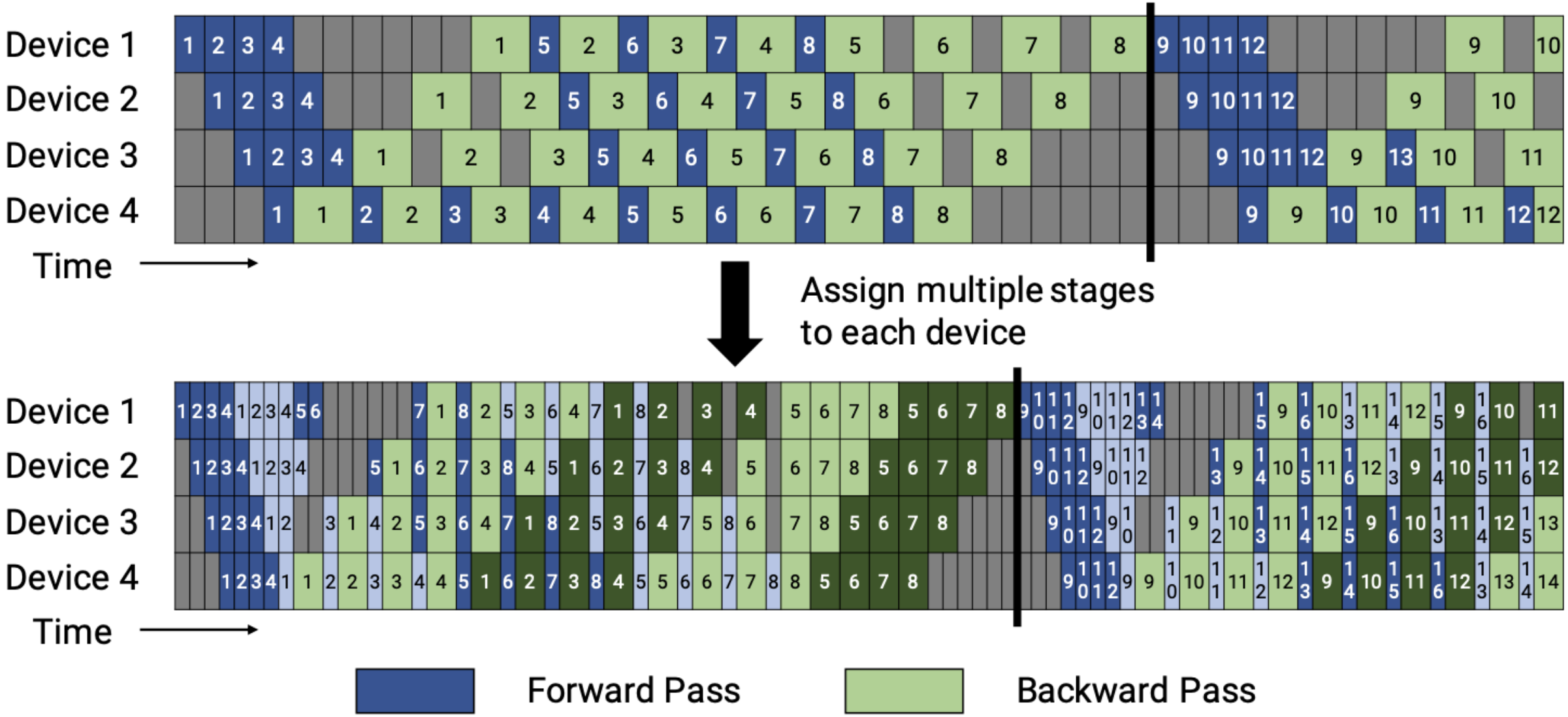
Bubble in GPipe

- Number of microbatches in a batch m
- Number of pipeline stages p
- Ideal time per iteration as Tid
- Time to execute a single microbatch's forward and backward pass as Tf and Tb .

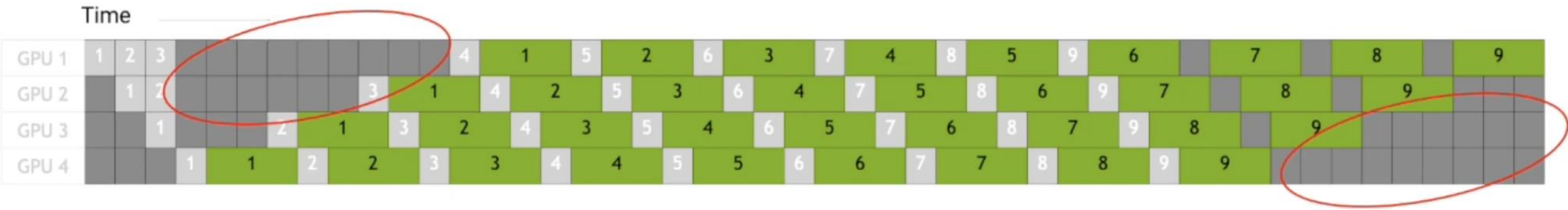
- Tpb = (p-1)(Tf + Tb)
- Tid = m(Tf+Tb)
- Bubble faction = (p-1)/m

This is called 1 forward 1 backward (1F1B)

Question: How is this better than GPipe?



PIPELINE BUBBLES



p : number of pipeline stages

m : number of micro batches

t_f : forward step time

t_b : backward step time

$$\text{total time} = (m + p - 1) \times (t_f + t_b)$$

$$\text{ideal time} = m \times (t_f + t_b)$$

$$\text{bubble time} = (p - 1) \times (t_f + t_b)$$

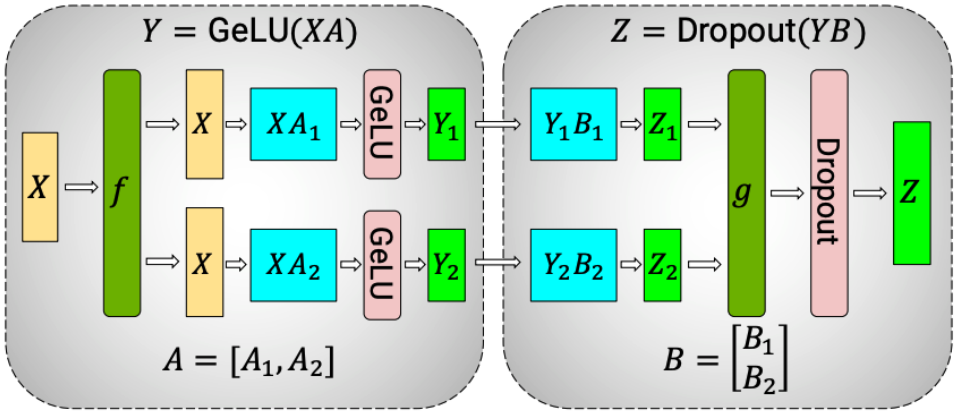
$$\text{bubble time overhead} = \frac{\text{bubble time}}{\text{ideal time}} = \frac{p - 1}{m}$$

Performance analysis

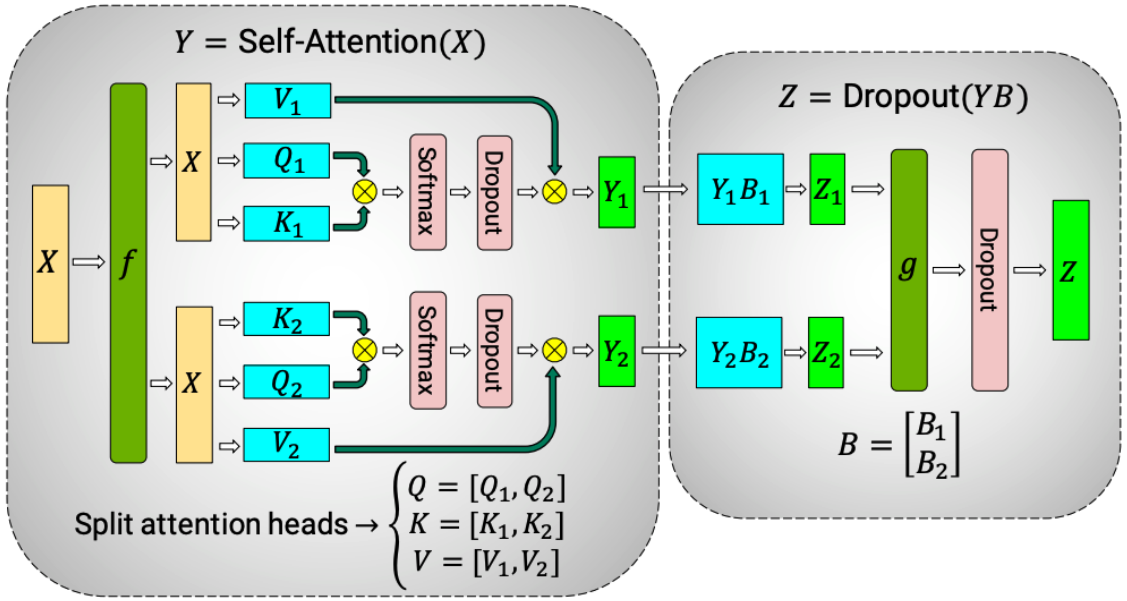
We use the following notation in this section:

- (p, t, d): Parallelization dimensions. p for the pipeline-model-parallel size, t for the tensor-model-parallel size, and d for the data-parallel size.
- n: Number of GPUs. We require $p \cdot t \cdot d = n$.
- B: Global batch size (provided as input).
- b: Microbatch size.
- $m = \frac{1}{b} \cdot \frac{B}{d}$: Number of microbatches in a batch *per pipeline*.

Now, let’s consider TP + MP



(a) MLP.



(b) Self-Attention.

Figure 5: Blocks of transformer model partitioned with tensor model parallelism (figures borrowed from Megatron [40]). f and g are conjugate. f is the identity operator in the forward pass and all-reduce in the backward pass, while g is the reverse.

Tensor and pipeline model parallelism can both be used to partition a model's parameters over multiple GPUs. As stated earlier, using pipeline parallelism with periodic flushes results in a pipeline bubble of size $(p-1)/m$. Let us assume that $d = 1$ (data-parallel size); consequently, $t \cdot p = n$. The pipeline bubble size in terms of t is:

$$\frac{p-1}{m} = \frac{n/t - 1}{m}.$$

As t increases, the pipeline bubble thus decreases for fixed B, b, and d ($m = B/(b \cdot d)$ is fixed as well).

Tensor and pipeline model parallelism can both be used to partition a model's parameters over multiple GPUs. As stated earlier, using pipeline parallelism with periodic flushes results in a pipeline bubble of size $(p-1)/m$. Let us assume that $d = 1$ (data-parallel size); consequently, $t \cdot p = n$. The pipeline bubble size in terms of t is:

$$\frac{p-1}{m} = \boxed{\frac{n/t - 1}{m}}.$$

As t increases, the pipeline bubble thus decreases for fixed B, b, and d ($m = B/(b \cdot d)$ is fixed as well).

The amount of communication performed between different GPUs is also affected by the values of p and t. Pipeline model parallelism features cheaper point-to-point communication. Tensor model parallelism, on the other hand, uses all-reduce communication (two all-reduce operations each in the forward and backward pass, see §2.3). With pipeline parallelism, the total amount of communication that needs to be performed between every pair of consecutive devices (for either the forward or backward pass) for each microbatch is bsh, where s is the sequence length and h is the hidden size. With tensor model parallelism, tensors of total size bsh need to be all-reduced among t model replicas twice each in the forward and backward pass for each layer, leading to a total communication of $8bsh\left(\frac{t-1}{t}\right)$ *per layer* per device for each microbatch. Each device typically has multiple layers; the total amount of tensor-parallel-communication per device for each microbatch is then $l^{\text{stage}} \cdot \left(8bsh\left(\frac{t-1}{t}\right)\right)$, where l^{stage} is the number of layers in a pipeline stage.

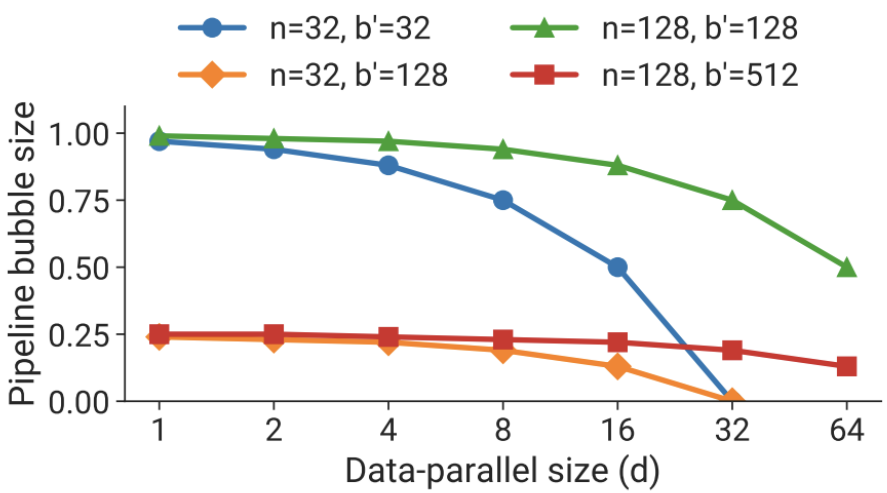
The amount of communication performed between different GPUs is also affected by the values of p and t. Pipeline model parallelism features cheaper point-to-point communication. Tensor model parallelism, on the other hand, uses all-reduce communication (two all-reduce operations each in the forward and backward pass, see §2.3). With pipeline parallelism, the total amount of communication that needs to be performed between every pair of consecutive devices (for either the forward or backward pass) for each microbatch is bsh, where s is the sequence length and h is the hidden size. With tensor model parallelism, tensors of total size bsh need to be all-reduced among t model replicas twice each in the forward and backward pass for each layer, leading to a total communication of $8bsh\left(\frac{t-1}{t}\right)$ *per layer* per device for each microbatch. Each device typically has multiple layers; the total amount of tensor-parallel-communication per device for each microbatch is then $l^{\text{stage}} \cdot \left(8bsh\left(\frac{t-1}{t}\right)\right)$, where l^{stage} is the number of layers in a pipeline stage.

Takeaway #1: When considering different forms of model parallelism, tensor model parallelism should generally be used up to degree g when using g-GPU servers, and then pipeline model parallelism can be used to scale up to larger models across servers.

Now, let’s consider DP + PP

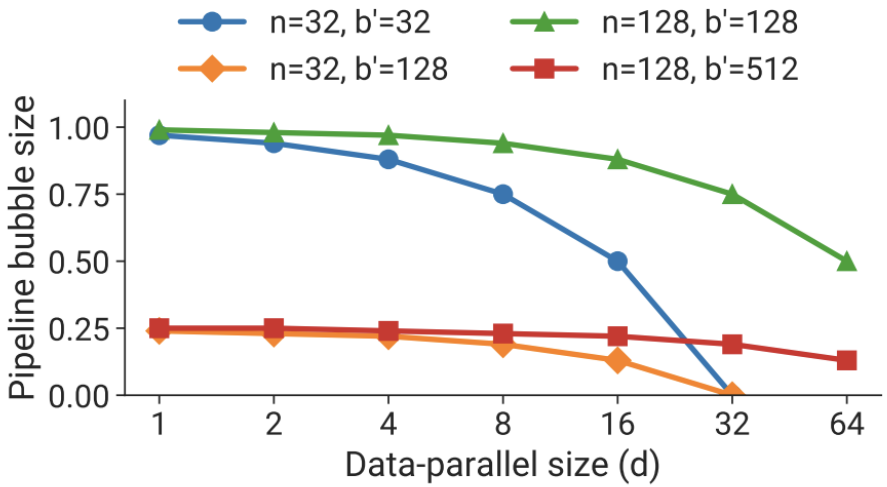
3.3.1 Pipeline Model Parallelism. Let $t = 1$ (tensor-model-parallel size). The number of microbatches per pipeline is $m = B/(d \cdot b) = b'/d$, where $b' := B/b$. With total number of GPUs n, the number of pipeline stages is $p = n/(t \cdot d) = n/d$. The pipeline bubble size is:

$$\frac{p-1}{m} = \frac{n/d - 1}{b'/d} = \frac{n-d}{b'}.$$



3.3.1 Pipeline Model Parallelism. Let $t = 1$ (tensor-model-parallel size). The number of microbatches per pipeline is $m = B/(d \cdot b) = b'/d$, where $b' := B/b$. With total number of GPUs n, the number of pipeline stages is $p = n/(t \cdot d) = n/d$. The pipeline bubble size is:

$$\frac{p-1}{m} = \frac{n/d - 1}{b'/d} = \boxed{\frac{n-d}{b'}}.$$



As d becomes larger, $n - d$ becomes smaller, and thus the pipeline bubble becomes smaller. Figure 6 shows the behavior of the pipeline bubble size for various values of d, n, and b'. It might not be possible to increase d all the way to n for all models, since a model's full training memory footprint might be larger than the memory capacity of a single accelerator.

Overall throughput will thus increase if the all-reduce communication needed for data parallelism does not drastically increase with higher d, which should hold since the communication time for a ring-based implementation scales with $\frac{d-1}{d} = 1 - \frac{1}{d}$.

We can also analyze the impact of increasing the batch size B. For a given parallel configuration, as the batch size B increases, $b' = B/b$ increases, $(n - d)/b'$ decreases, consequently increasing throughput. All-reduce communication required by data parallelism also becomes more infrequent, further increasing throughput.

Now, let’s consider DP + MP

3.3.2 Data and Tensor Model Parallelism. With tensor model parallelism, all-reduce communication needs to be performed for *every* microbatch. This can be expensive across multi-GPU servers. On the other hand, data parallelism only needs to perform expensive all-reduce communication *once per batch*. Moreover, with tensor model parallelism, each model-parallel rank performs a subset of the computation in each model layer, and thus for insufficiently-large layers, modern GPUs might not perform these sub-matrix computations with peak efficiency.

Takeaway #2: When using data and model parallelism, a total model-parallel size of $M = t \cdot p$ should be used so that the model's parameters and intermediate metadata fit in GPU memory; data parallelism can be used to scale up training to more GPUs.

Now, let’s determine microbatch size

3.4 Microbatch Size

The choice of the microbatch size b also affects model-training throughput. For example, we see in Figure 7 that per-GPU throughput increases by up to 1.3× with a larger microbatch size on a single GPU. We now want to determine the optimal microbatch size b given a parallel configuration (p, t, d) and batch size B. The amount of data-parallel communication will be the same regardless of the microbatch size. Given functions $t_f(b)$ and $t_b(b)$ that map the microbatch size to the forward and backward computation times for a single microbatch, the total time spent computing a batch, ignoring communication cost, is (as before, define b' as B/d):

$$\left(b'/b + p - 1\right) \cdot \left(t_f(b) + t_b(b)\right). \tag{1}$$

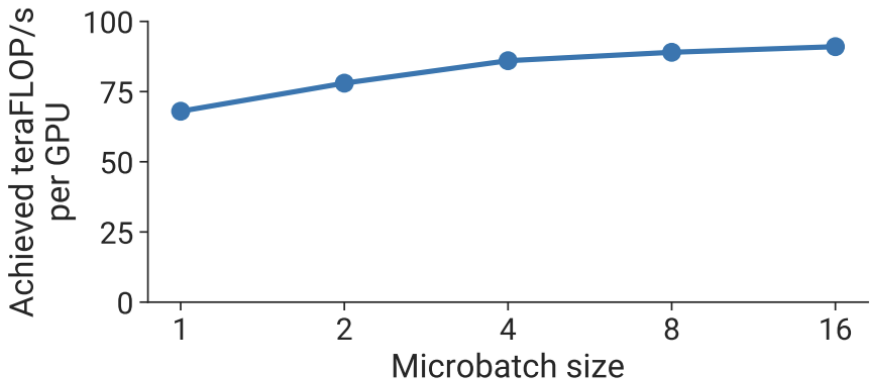
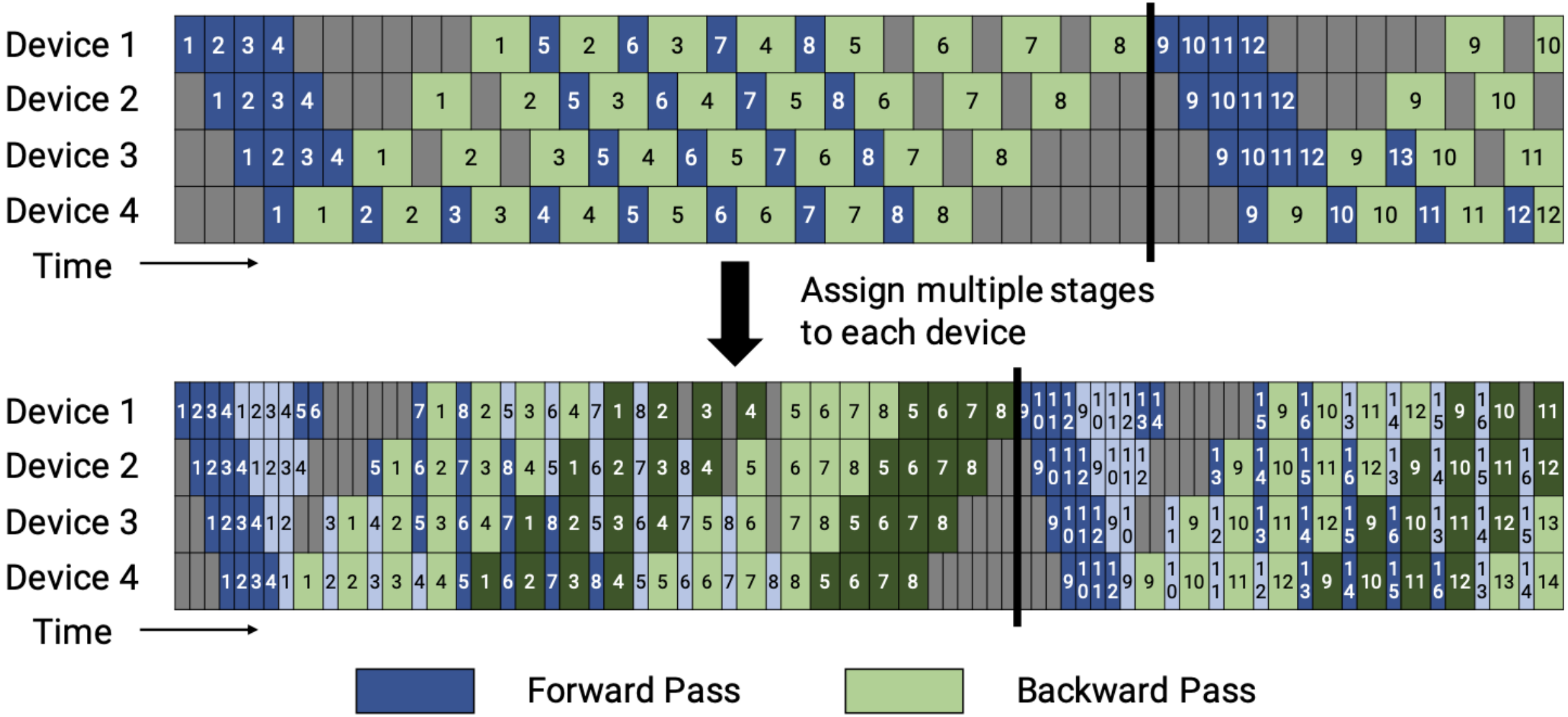


Figure 7: Per-GPU throughput versus microbatch size for a GPT model with a billion parameters (128 attention heads, hidden size of 4096, 4 transformer layers).

This is called 1 forward 1 backward (1F1B)

Question: How is this better than GPipe?



3.4 Microbatch Size

The choice of the microbatch size b also affects model-training throughput. For example, we see in Figure 7 that per-GPU throughput increases by up to 1.3× with a larger microbatch size on a single GPU. We now want to determine the optimal microbatch size b given a parallel configuration (p, t, d) and batch size B. The amount of data-parallel communication will be the same regardless of the microbatch size. Given functions $t_f(b)$ and $t_b(b)$ that map the microbatch size to the forward and backward computation times for a single microbatch, the total time spent computing a batch, ignoring communication cost, is (as before, define b' as B/d):

$$(b'/b + p - 1) \cdot \Big(t_f(b) + t_b(b)\Big). \tag{1}$$

The microbatch size thus affects both the arithmetic intensity of operations as well as the pipeline bubble size (by affecting m). Figure 8 shows estimated throughput (equation (1) used to estimate processing time) for a GPT model with a billion parameters and $(p, t) = (8, 8)$. The optimal b for both batch sizes is 4.

Takeaway #3: The optimal microbatch size b depends on the throughput and memory footprint characteristics of the model, as well as the pipeline depth p, data-parallel size d, and batch size B.

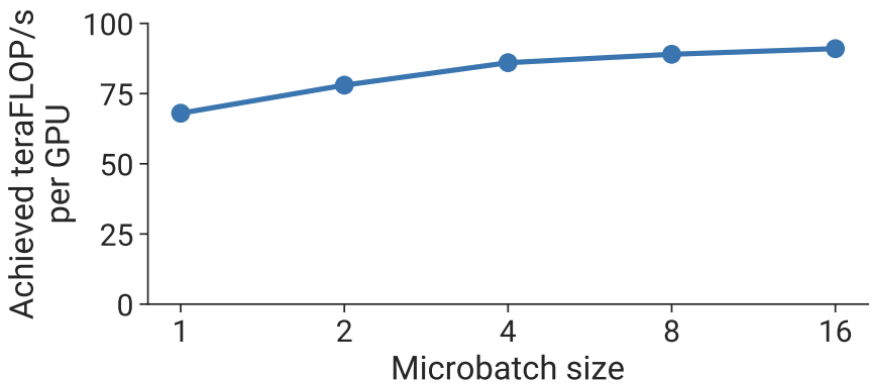


Figure 7: Per-GPU throughput versus microbatch size for a GPT model with a billion parameters (128 attention heads, hidden size of 4096, 4 transformer layers).

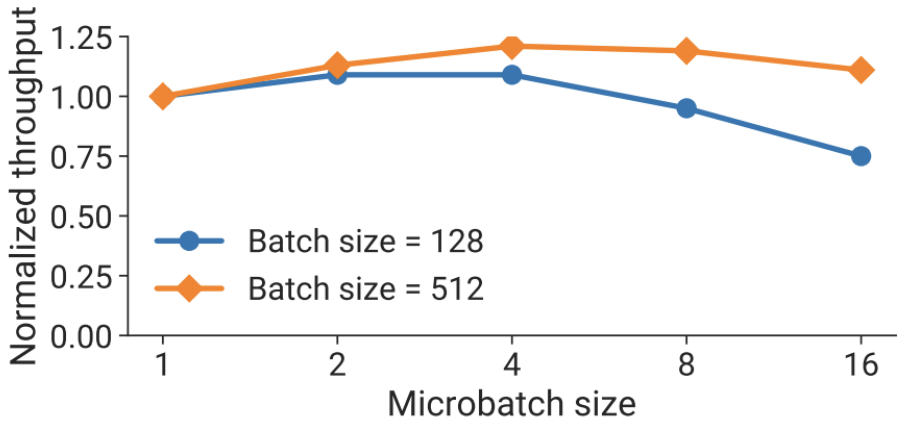


Figure 8: Behavior of normalized estimated throughput (time computed as $t = (b'/b + p - 1) \cdot \big(t_f(b) + t_b(b)\big)$) with respect to the microbatch size b for the same GPT model from Figure 7.

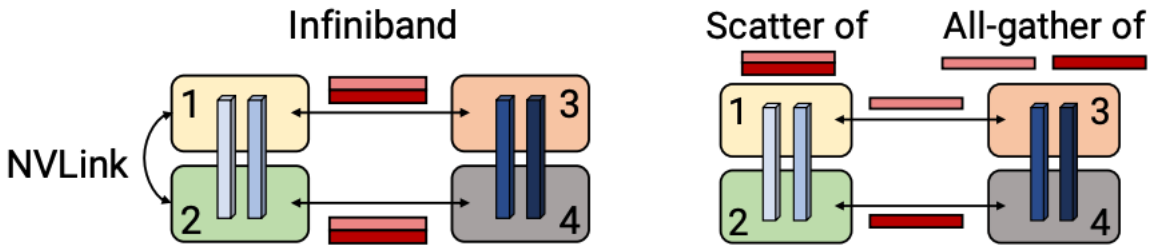
Now, let’s determine how to do activation recomputation

3.5 Activation Recomputation

Activation recomputation [12, 18, 20, 21] is an optional technique that trades off an increase in the number of compute operations performed for additional memory footprint, by running the forward pass a second time just before the backward pass (and stashing only the input activations for a given pipeline stage, as opposed to the entire set of intermediate activations, which is much larger). Activation recomputation is required to train reasonably large models with pipeline parallelism to keep memory footprint acceptably low. Previous work like PipeDream-2BW [30] has looked at the performance ramifications of activation recomputation.

The number of activation checkpoints does not impact throughput, but impacts memory footprint. Let A^{input} be the size of the input activations of a layer, and $A^{\text{intermediate}}$ be the size of intermediate activations per layer. If a model stage has l layers, and if c is the number of checkpoints, the total memory footprint is going to be $c \cdot A^{\text{input}} + l/c \cdot A^{\text{intermediate}}$. The minimum value of this function is obtained when $c = \sqrt{l \cdot \left(A^{\text{intermediate}}/A^{\text{input}}\right)}$. In practice, we measure $A^{\text{intermediate}}$ empirically. For most cases, checkpointing every 1 or 2 transformer layers is optimal.

Communication optimizations



(a) W/o scatter/gather optimization. (b) With scatter/gather optimization.

Figure 9: Scatter/gather communication optimization. Light blue blocks are layers in the first pipeline stage, and dark blue blocks are layers in the second pipeline stage. Without the scatter/gather optimization, the same tensor is sent redundantly over inter-node InfiniBand links. Instead, at the sender, we can scatter the tensor into smaller chunks, reducing the sizes of tensors sent over InfiniBand links. The final tensor can then be rematerialized at the receiver using a gather operation.

Number of parameters (billion)	Attention heads	Hidden size	Number of layers	Tensor model-parallel size	Pipeline model-parallel size	Number of GPUs	Batch size	Achieved teraFlOP/s per GPU	Percentage of theoretical peak FLOP/s	Achieved aggregate petaFLOP/s
1.7	24	2304	24	1	1	32	512	137	44%	4.4
3.6	32	3072	30	2	1	64	512	138	44%	8.8
7.5	32	4096	36	4	1	128	512	142	46%	18.2
18.4	48	6144	40	8	1	256	1024	135	43%	34.6
39.1	64	8192	48	8	2	512	1536	138	44%	70.8
76.1	80	10240	60	8	4	1024	1792	140	45%	143.8
145.6	96	12288	80	8	8	1536	2304	148	47%	227.1
310.1	128	16384	96	8	16	1920	2160	155	50%	297.4
529.6	128	20480	105	8	35	2520	2520	163	52%	410.2
1008.0	160	25600	128	8	64	3072	3072	163	52%	502.0

Table 1: Weak-scaling throughput for GPT models ranging from 1 billion to 1 trillion parameters.

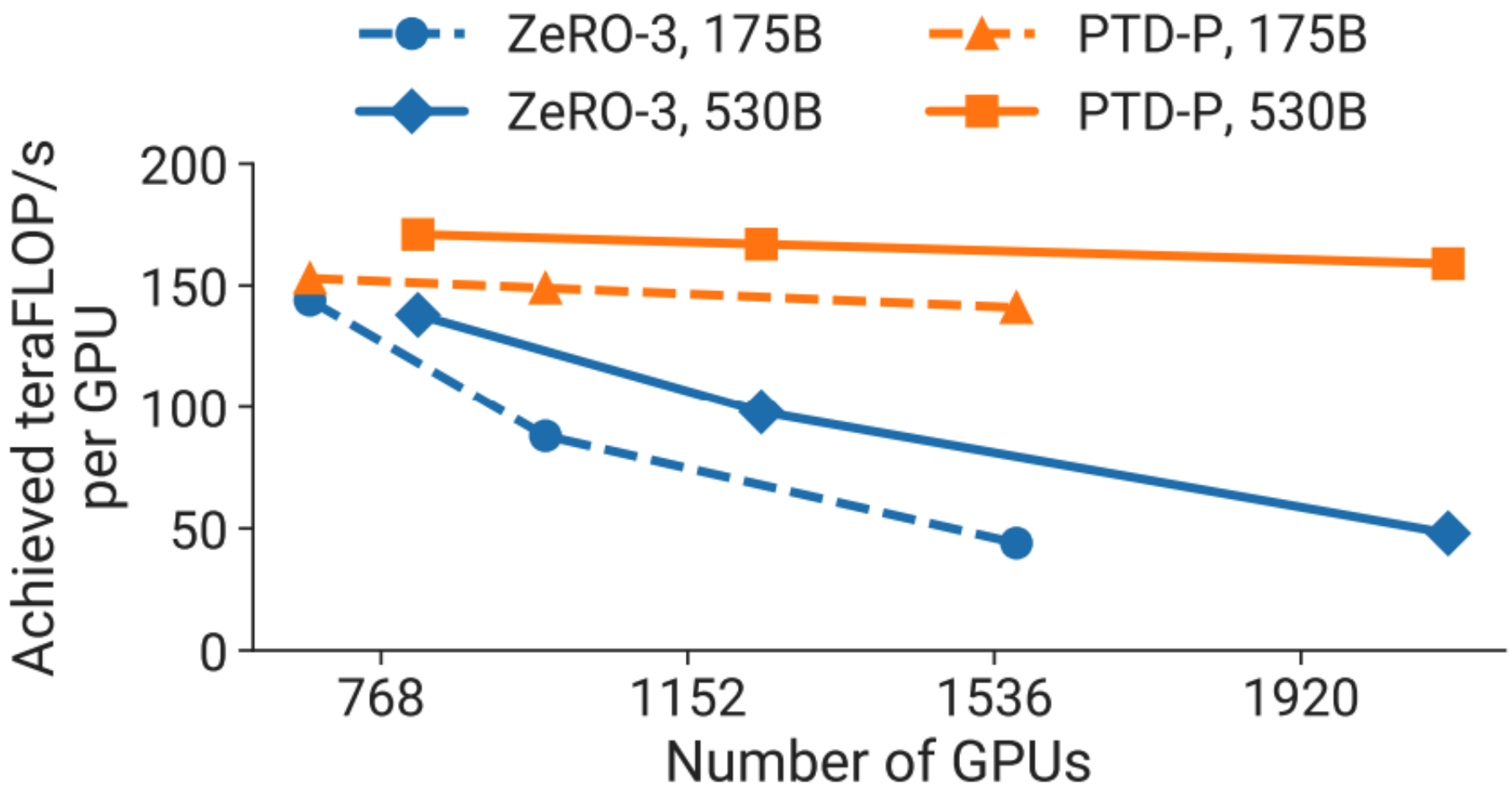


Figure 10: Throughput per GPU of PTD-P and ZeRO-3 for two different GPT models (the 175B GPT-3 model is shown with dotted lines, and the 530B model is shown with solid lines). Global batch sizes are fixed and ZeRO-3 is used without any model parallelism.

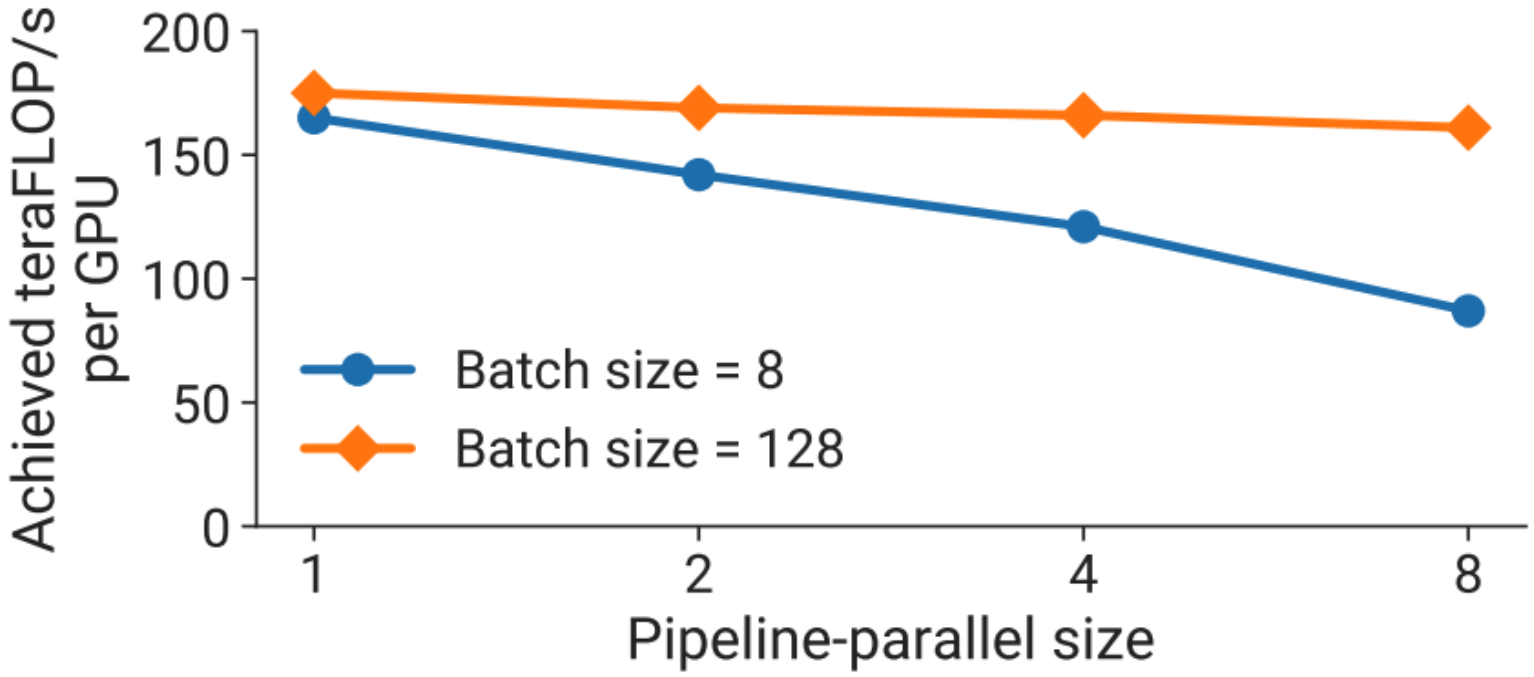


Figure 11: Throughput per GPU of pipeline parallelism using two different batch sizes in a weak-scaling experiment setup (model size increases with the pipeline-parallel size).

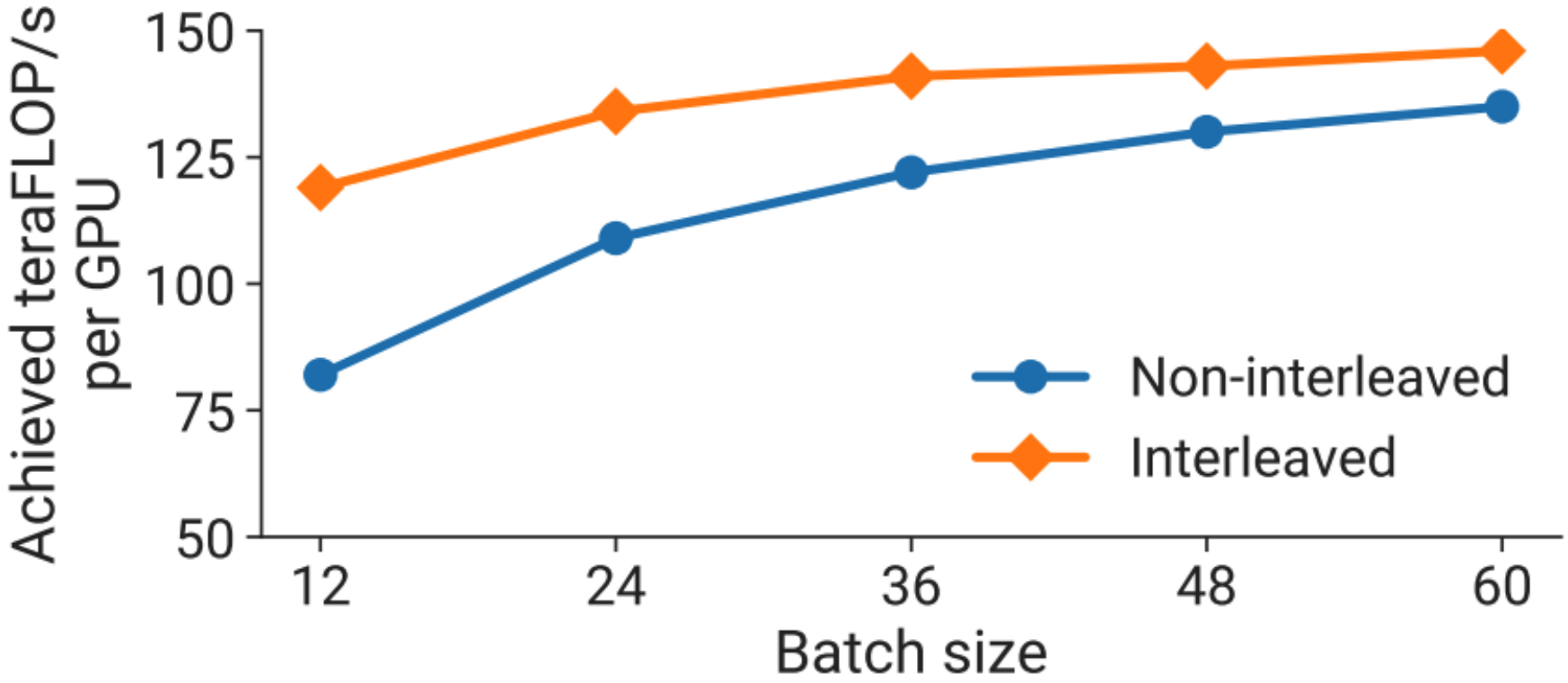


Figure 12: Throughput per GPU of interleaved and non-interleaved schedules for a GPT model (175 billion parameters) on 96 GPUs.

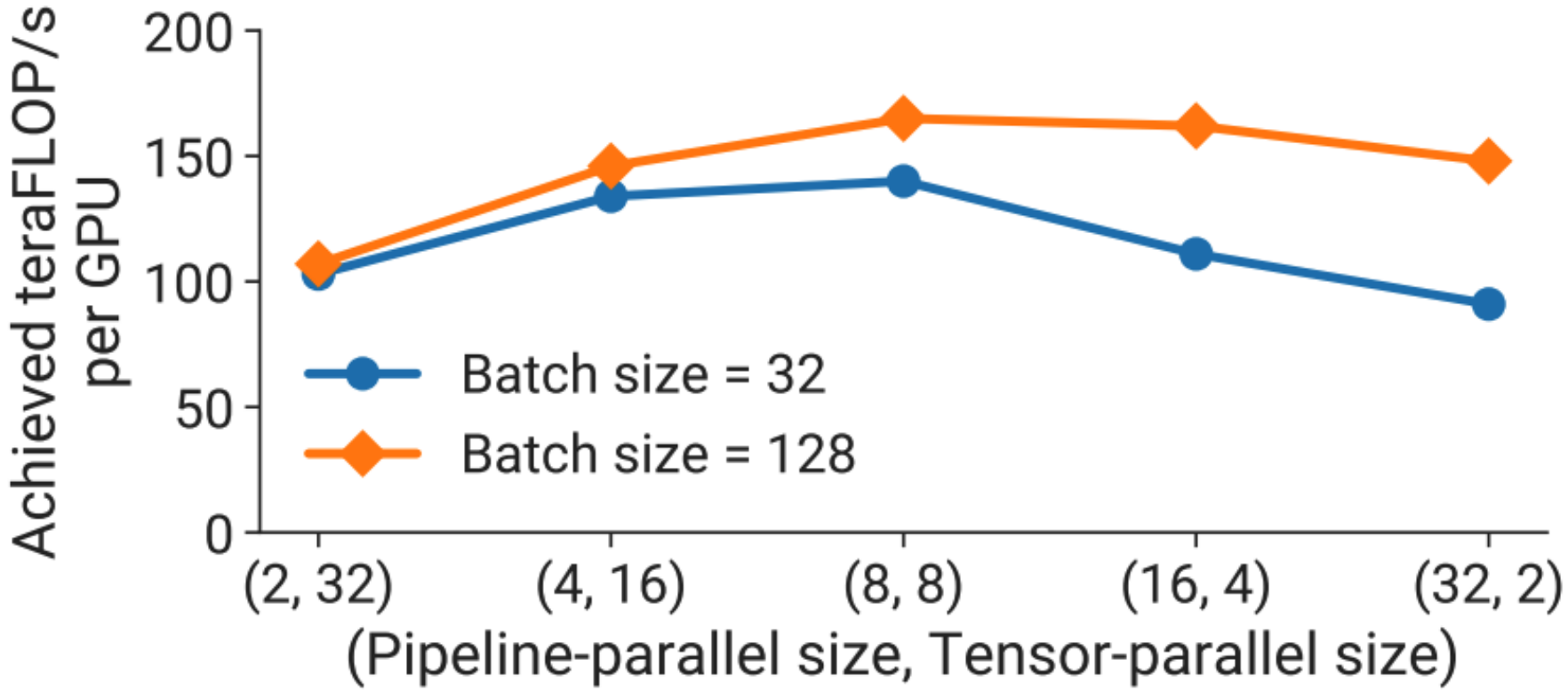


Figure 13: Throughput per GPU of various parallel configurations that combine pipeline and tensor model parallelism using a GPT model with 162.2 billion parameters and 64 A100 GPUs.

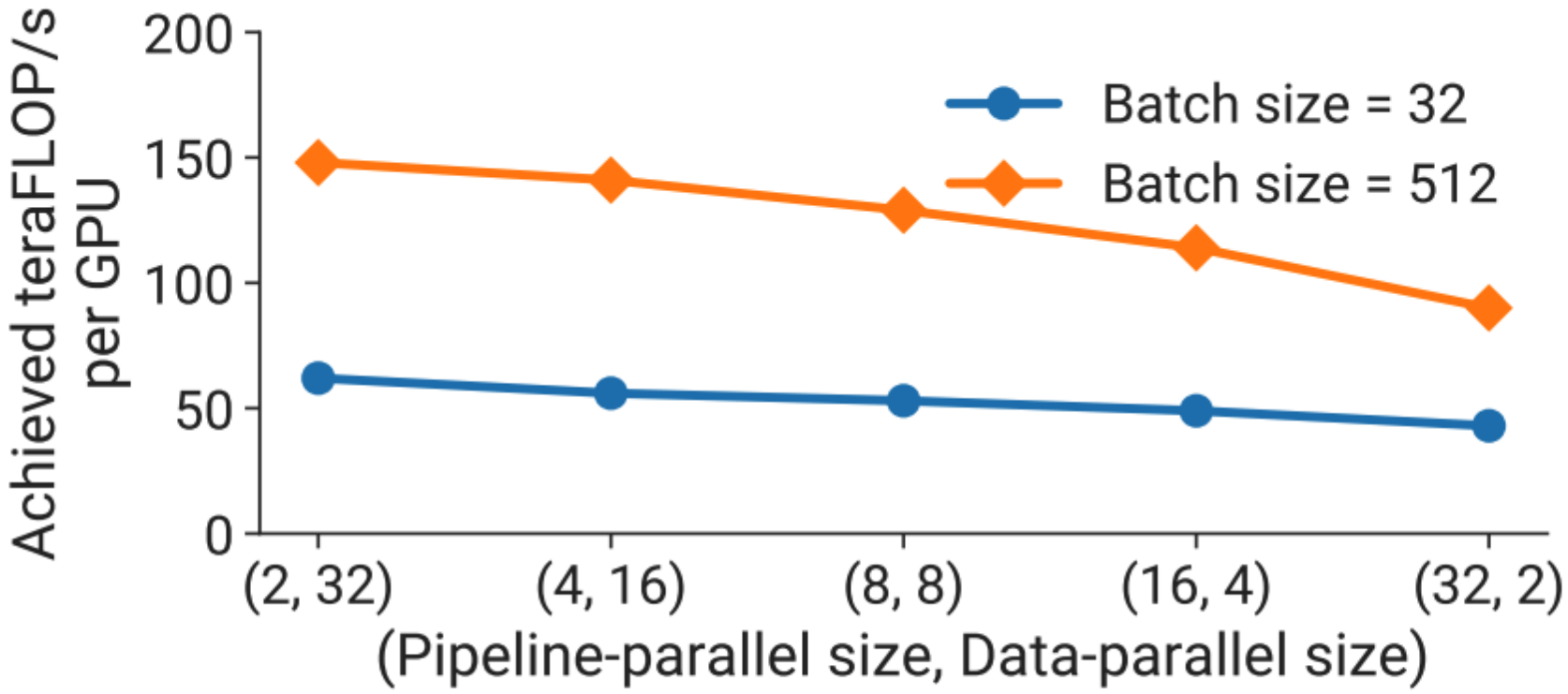


Figure 14: Throughput per GPU of various parallel configurations that combine data and pipeline model parallelism using a GPT model with 5.9 billion parameters, three different batch sizes, microbatch size of 1, and 64 A100 GPUs.

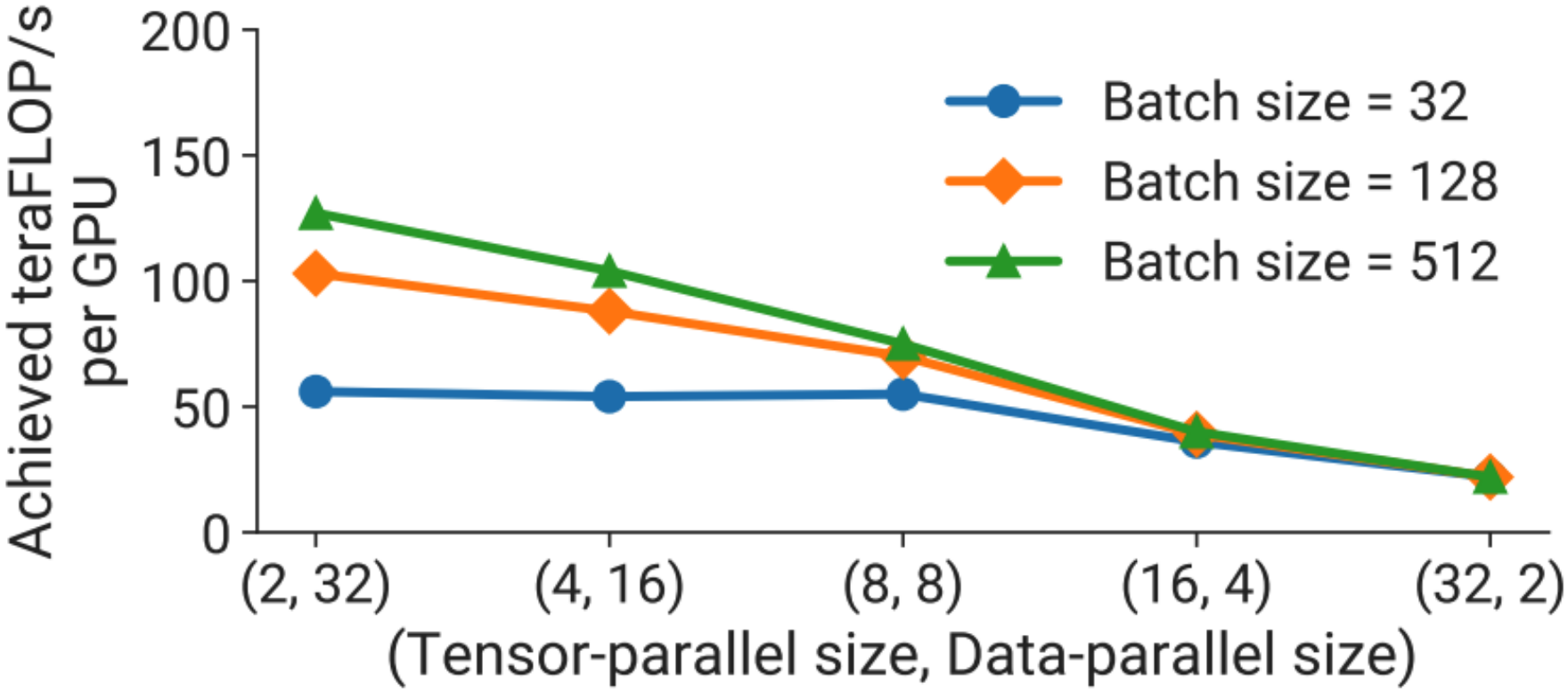


Figure 15: Throughput per GPU of various parallel configurations that combine data and tensor model parallelism using a GPT model with 5.9 billion parameters, three different batch sizes, microbatch size of 1, and 64 A100 GPUs.

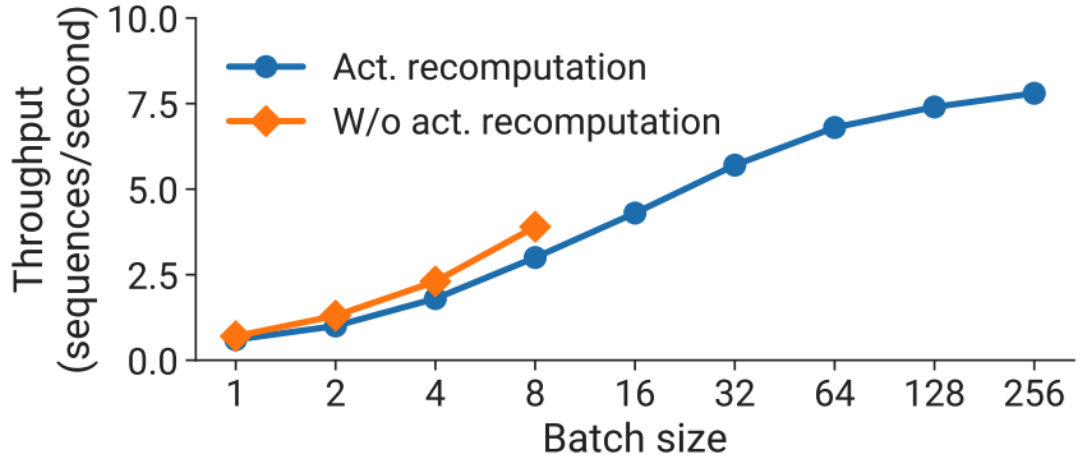


Figure 17: Throughput (in sequences per second) with and without activation recomputation for a GPT model with 145 billion parameters using 128 A100 GPUs ($(t, p) = (8, 16)$).

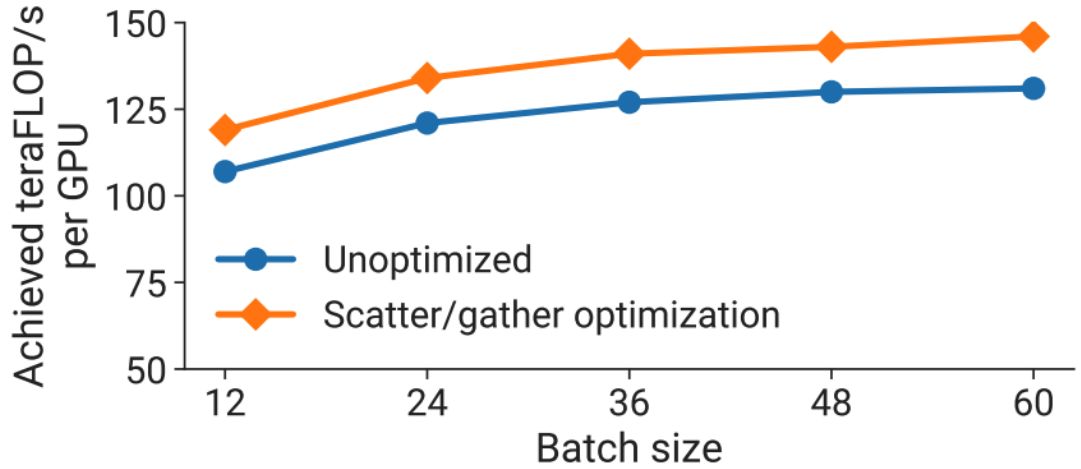
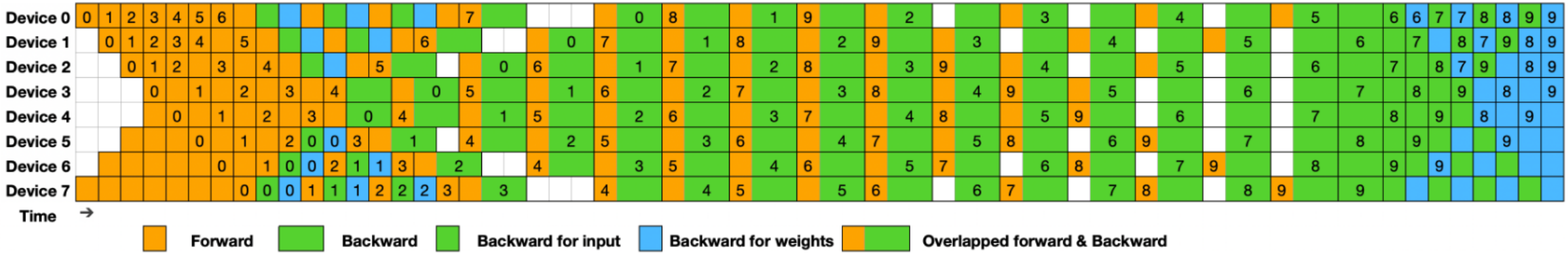


Figure 18: Throughput per GPU with and without the scatter/gather optimization for a GPT model with 175 billion parameters using 96 A100 GPUs and the interleaved schedule.

New PP design: DeepSeek DualPipe

Schedules

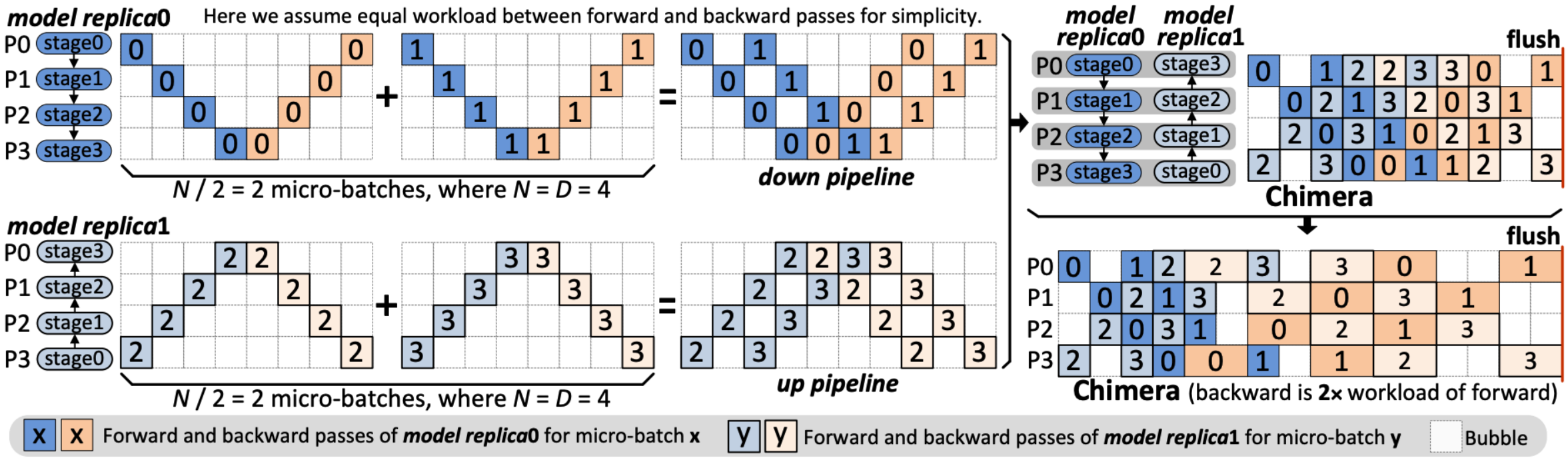


Example DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction, so we omit their batch ID for illustration simplicity. Two cells enclosed by a shared black border have mutually overlapped computation and communication

New PP design: DeepSeek DualPipe

Method	Bubble	Parameter Per Device	Activation Per Device	#Devices
1F1B	$(PP-1)(F+B)$	1×	PP	PP
ZB1P	$(PP-1)(F+B-2W)$	1×	PP	PP
DualPipe	$(PP/2-1)(F\&B+B-3W)$	2×	$PP+1$	PP
DualPipeV	$(PP/2-1)(F\&B+B-3W)$	2×	$PP+1$	$PP/2$

Chimera


model replica0
Here we assume equal workload between forward and backward passes for simplicity.
P0 stage0
P1 stage1
P2 stage2
P3 stage3
N / 2 = 2 micro-batches, where N = D = 4
down pipeline
model replica1
P0 stage3
P1 stage2
P2 stage1
P3 stage0
N / 2 = 2 micro-batches, where N = D = 4
up pipeline
model replica0
model replica1
P0 stage0 stage3
P1 stage1 stage2
P2 stage2 stage1
P3 stage3 stage0
flush
Chimera
flush
Chimera (backward is 2× workload of forward)
Forward and backward passes of model replica0 for micro-batch x
Forward and backward passes of model replica1 for micro-batch y
Bubble

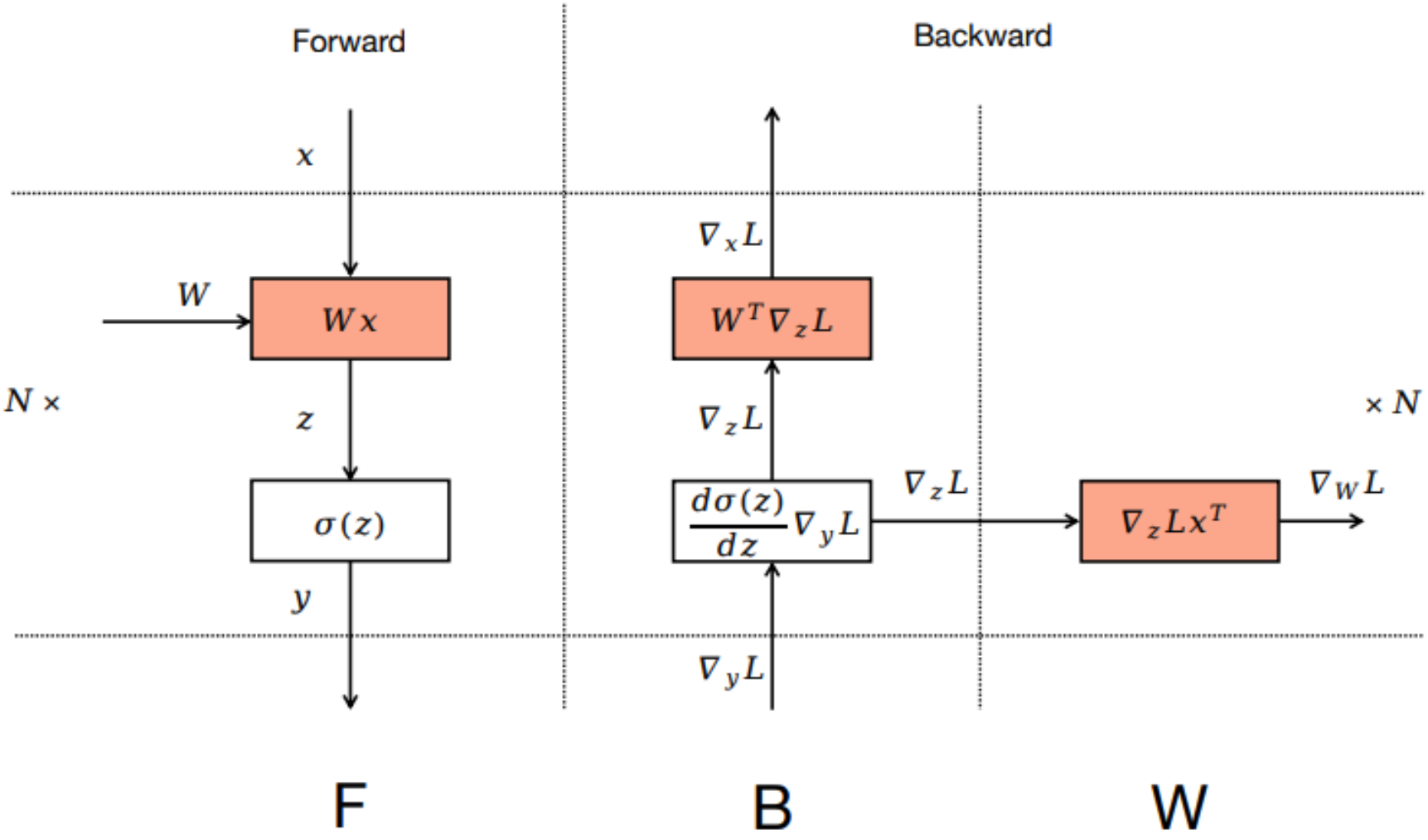


Figure 1: Computation Graph for MLP.

Zero Bubble Pipeline Parallelism

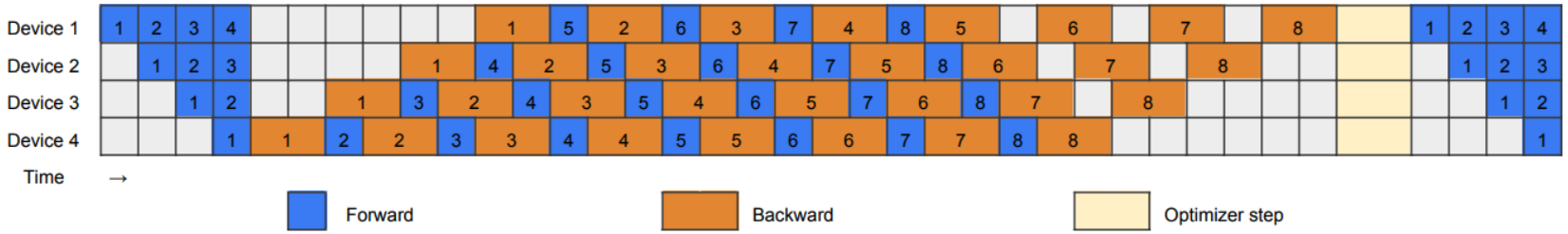


Figure 2: 1F1B pipeline schedule.

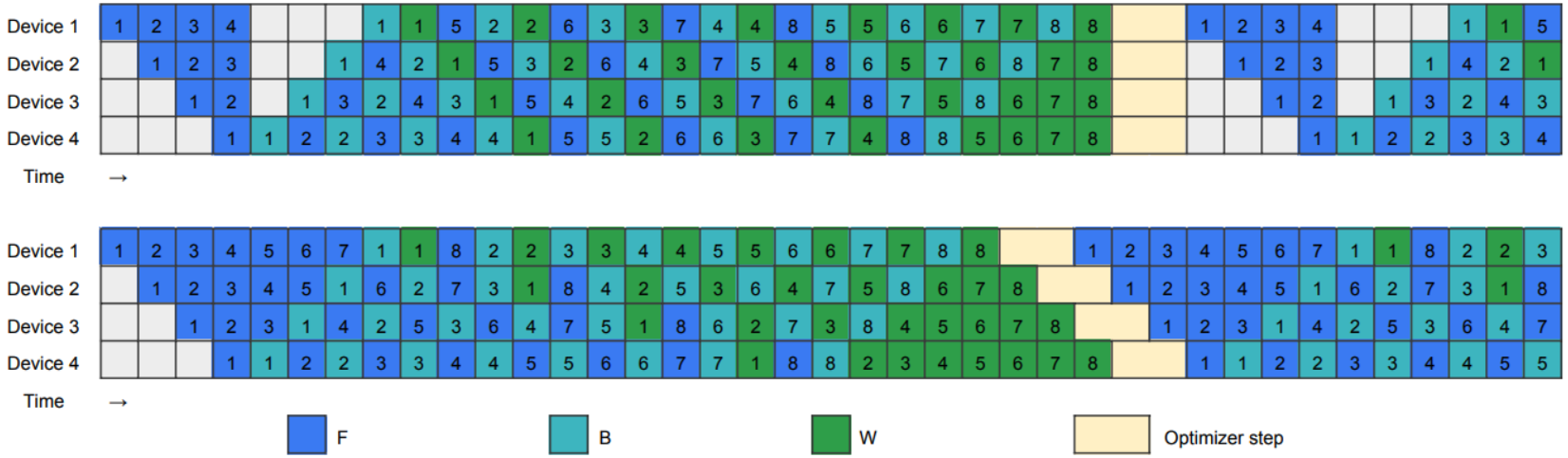


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

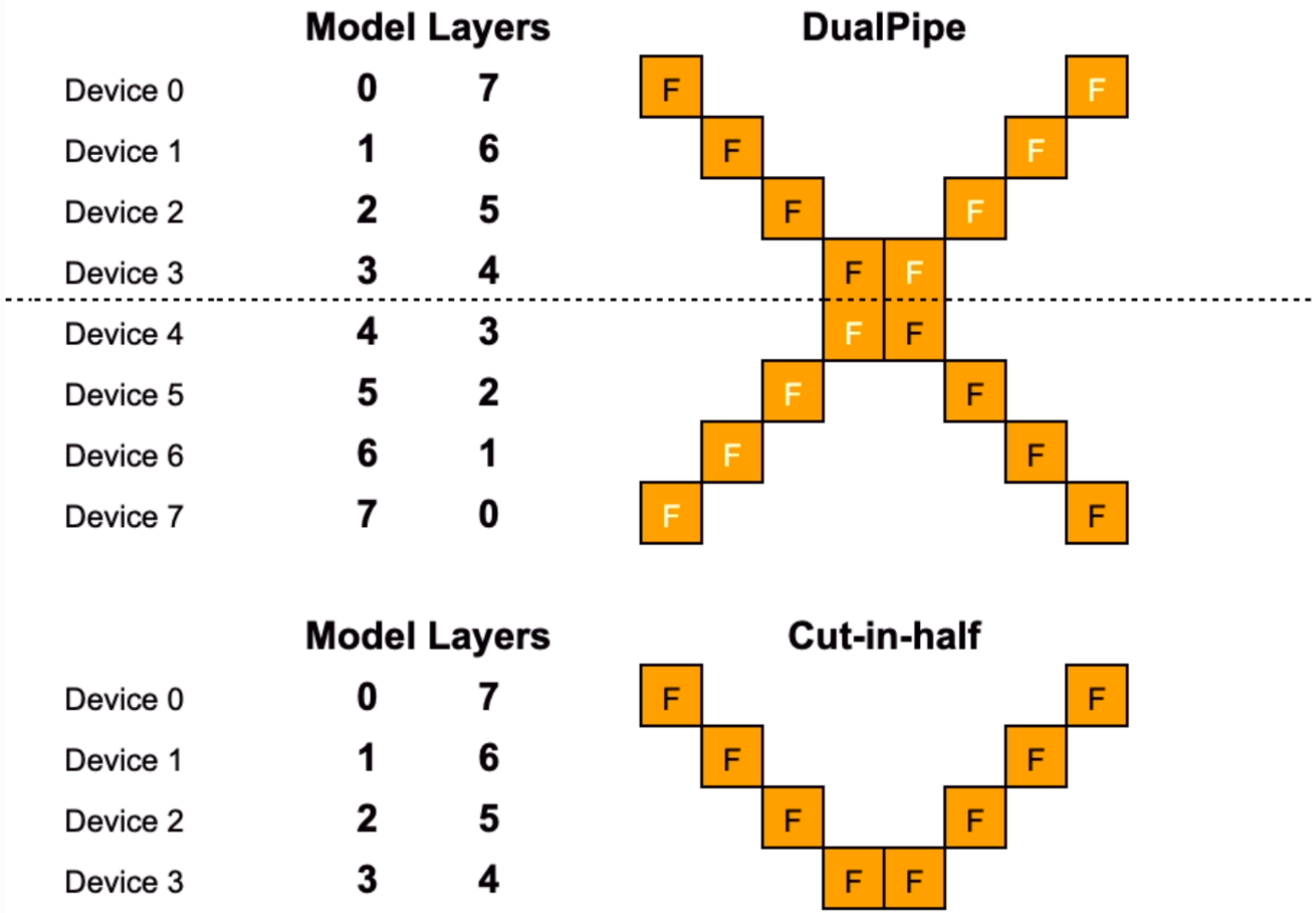


Figure 2. Illustration of halfing the devices & grafting the stages.

Figure 3. The Cut-in-half schedule.